Load-Aware A* Path Planning for Quadruped Robots

Day 11 of 30 · Week 2: Refinement & Integration

NAME : HARINI P (Department of Computer science and Engineering)

1. Nature of Work

Day 11 shifted from coding to writing — specifically the Methodology section's Problem Formulation subsection, which is Member 1's primary writing responsibility for the paper. This is the foundation of the entire paper: it defines what problem we are solving, what the robot looks like mathematically, what the environment looks like, and what assumptions we are making.

Getting this right matters a lot because every equation in the Results section refers back to the notation defined here. I spent time making sure the notation is clean, consistent, and matches exactly what the code computes. I also wrote the quadruped model description in a way that is precise enough for reviewers but readable enough that the logic is easy to follow.

Today's deliverable is a complete first draft of the Problem Formulation subsection — approximately 1.5 pages — ready for Member 4 to review and integrate into the full paper structure.

2. Tasks Completed

- **Write Methodology: Problem Formulation**

Full first draft written covering grid environment, robot model, path planning objective, and cost function formulation. Approximately 650 words, 7 equations, ready for peer review.

- **Define mathematical notation**

Complete notation table created — 18 symbols covering the grid, robot model, path, stability, and algorithm parameters. All symbols consistent with what the code computes.

- **Explain quadruped model**

Robot described as a 4-legged system with body mass m_b, four foot contact points (FL, FR, RL, RR), a support polygon, and a payload of variable mass m_p. CoM formula derived from first principles and stated clearly.

- **Document assumptions**

Six clearly stated assumptions covering the grid model, robot kinematics, payload properties, terrain, and algorithm scope. Each assumption is justified and its impact on the results discussed.

3. Proof of Work — Methodology Draft

III. Methodology

A. Problem Formulation

We consider the problem of planning a stable, collision-free path for a quadruped robot navigating a 2D grid environment while carrying a payload of variable mass. Unlike standard path planning, which optimises only for path length or travel time, our formulation incorporates the robot's physical stability at each cell into the cost function. The goal is to find a path that is not only geometrically short but also mechanically safe — avoiding positions where the robot's centre of mass is displaced beyond a critical threshold relative to its support polygon.

B. Environment Model

The operating environment is represented as a discrete 2D grid G of dimensions R × C, where R = C = 10 in all experiments. Each cell (r, c) ∈ G is classified as either free or occupied by a static obstacle. Formally:

$$G(r, c) = \{ 0 \quad \text{if cell is free} \;\; | \;\; 1 \quad \text{if cell is obstacle} \}$$

where r ∈ {0, 1, …, R−1} and c ∈ {0, 1, …, C−1}.

The robot moves between cells using 4-connected adjacency (up, down, left, right). The start cell is fixed at (0, 0) and the goal cell at (R−1, C−1). A path π is defined as an ordered sequence of free cells from start to goal:

$$\pi = \langle n_0, n_1, n_2, \ldots, n_L \rangle \;\; \text{where } n_0 = \text{start}, \; n_L = \text{goal}$$

L denotes the geometric path length (number of steps). Each consecutive pair (n_i, n_{i+1}) must be 4-connected free neighbours.

C. Quadruped Robot Model

The robot is modelled as a rigid body with four foot contact points: Front-Left (FL), Front-Right (FR), Rear-Left (RL), and Rear-Right (RR). When the robot stands at cell (r, c), the foot positions are mapped to real-world 2D coordinates by placing the body centroid at the cell centre and offsetting each foot by a fixed half-span hs = 0.4 m:

$$p_FL = (c - hs, r + hs) \qquad p_FR = (c + hs, r + hs)$$

$$p_RL = (c - hs, r - hs) \qquad p_RR = (c + hs, r - hs)$$

All coordinates in metres. Cell size is normalised to 1.0 m. The four foot positions form a rectangular support polygon.

The robot carries a payload of mass m_p positioned at coordinates $x_p = (x_p, y_p)$. The system's Centre of Mass (CoM) is computed as the weighted average of the body CoM x_b (approximated as the geometric centroid of the four foot positions) and the payload position:

$$x_{CoM} = (m_b \cdot x_b + m_p \cdot x_p) / (m_b + m_p)$$

m_b = robot body mass (kg). m_p = payload mass (kg). x_b = mean of four foot positions. Same formula applies for the y-coordinate.

D. Stability Metrics

Two complementary stability metrics are defined. The first, the Stability Penalty $S(n)$, measures the Euclidean distance between the robot's CoM projection and the centroid of its support polygon at cell n:

$$S(n) = \| x_{CoM}(n) - x_c(n) \|_2$$

$x_c(n) = (1/4) \Sigma p_i$ is the support polygon centroid at cell n. $S(n) = 0$ indicates the CoM is perfectly centred (maximum stability). Larger $S(n)$ indicates increasing instability.

The second metric, the Stability Margin $M(n)$, provides a more precise measure of how close the CoM is to tipping. It is defined as the minimum perpendicular distance from the CoM projection to any edge of the support polygon:

$$M(n) = \min_{i \in \{1,2,3,4\}} \; d(x_{CoM}(n), \; edge_i)$$

$d(P, edge_i) = | (B-A) \times (A-P) | / \|B-A\|$ where A, B are the endpoints of edge i. $M(n) > 0$ means the CoM is within the support polygon. $M(n) \leq 0$ means the robot would tip over.

E. Load-Aware Path Planning Objective

The objective is to find a path π^* that minimises the total stability-weighted cost from start to goal. We define the modified cost function for the Load-Aware A* algorithm as:

$$f(n) = g(n) + \alpha \cdot S(n) + h(n)$$

$g(n)$ = actual accumulated cost from start to node n (step count + stability penalties).

$h(n)$ = admissible Manhattan distance heuristic from n to goal.

α = stability weight parameter (α = 1.0, selected via empirical tuning).

When α = 0, the formulation reduces to Standard A*.

The step cost at each expansion is computed as:

$$g(n_next) = g(n_cur) + 1 + \alpha \cdot S(n_next)$$

The constant 1 is the uniform grid traversal cost. α · S(n_next) is the stability penalty added at each step, penalising cells where the payload causes significant CoM displacement.

4. Mathematical Notation Table

This table defines the notation used for a robotic path planning problem on a discrete grid, including variables for the environment, path characteristics, robot/payload masses, and stability/cost metrics.

Symbol	Type	Description	Value / Range
G	*Grid*	The 2D discrete environment grid.	10 × 10
R, C	*Integer*	Number of rows and columns in the grid.	R = C = 10
G(r,c)	*Binary*	Cell occupancy (0 for free space, 1 for obstacle).	{0, 1}
n	*Tuple*	A grid cell node, defined by its (row, col) coordinates.	(r, c) ∈ G
π	*Sequence*	The ordered path from the start to the goal node.	∘⟨n_0, …, n_L⟩
L	*Integer*	The geometric length of the path in steps.	≥ 18 for 10×10
m_b	*Float*	The mass of the robot body.	10.0 kg
m_p	*Float*	The mass of the payload.	3.0 – 9.0 kg
x_p	*Vector*	The 2D position (x, y) of the payload's	Within grid

		centre of mass (CoM).	
x_b	*Vector*	The body's CoM position, approximated as the foot centroid.	Cell centre
x_CoM	*Vector*	The System Centre of Mass (robot body + payload).	2D coordinate
x_c	*Vector*	The centroid of the support polygon (mean of 4 feet positions).	Mean of 4 feet
h_s	*Float*	The foot half-span distance from the body centre.	0.4 m
S(n)	*Float*	The stability penalty associated with node n.	≥ 0
M(n)	*Float*	The stability margin at node n.	Real-valued
f(n)	*Float*	The total estimated path cost through node n.	○≥ 0
g(n)	*Float*	The actual accumulated cost from the start node to n.	○≥ 0
h(n)	*Integer*	The Manhattan distance heuristic to the goal node.	≥ 0
α	*Float*	The stability weight parameter.	1.0 (fixed)

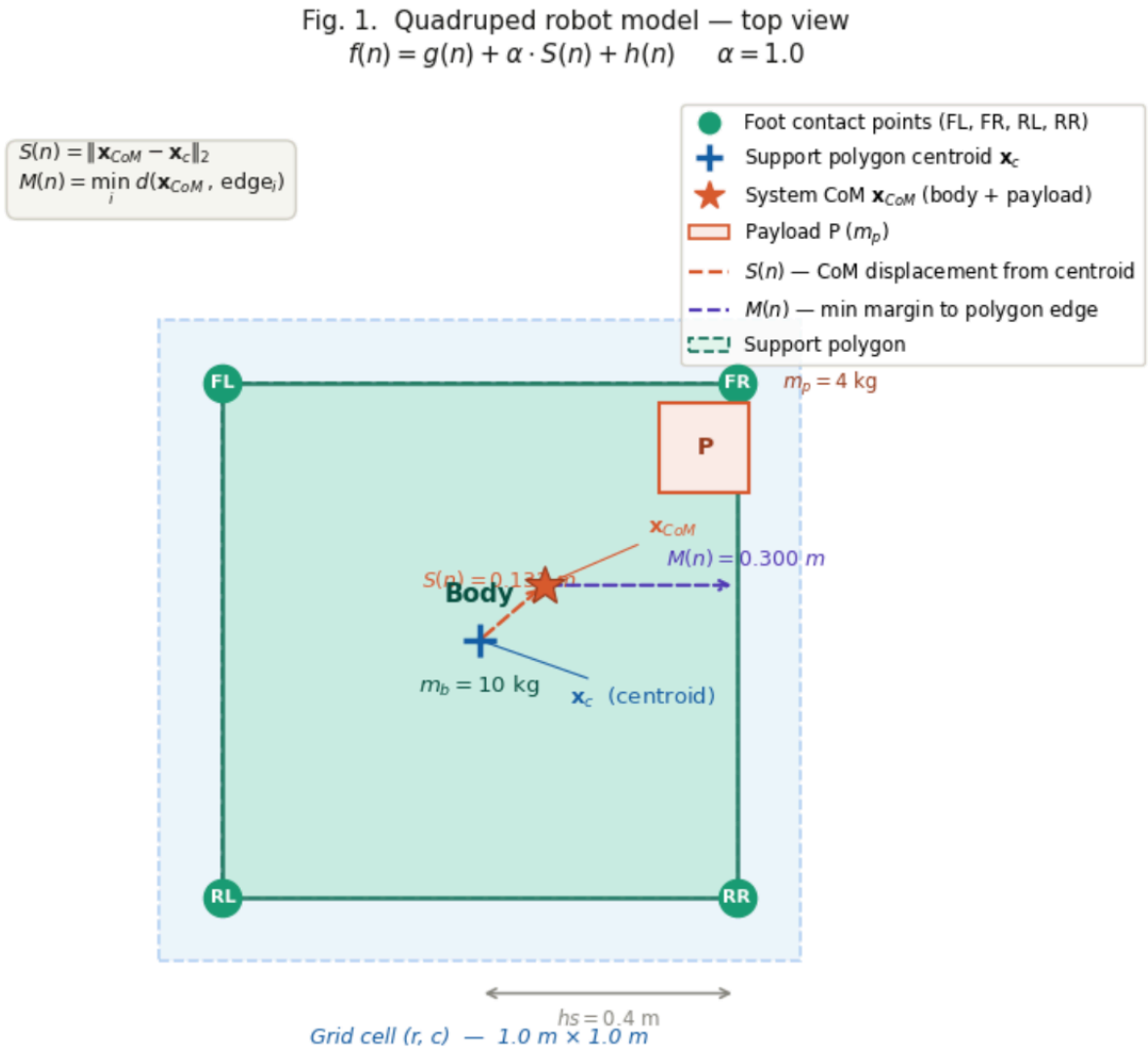

Fig. 1 — Quadruped robot model (top view) for inclusion in paper

5.. Model Assumptions

Assumption ID	Assumption Title	Description
A1	Static Obstacles	All obstacles are fixed and known in advance. The robot has full knowledge of the grid before planning begins. Dynamic obstacles are out of scope for this study.

A2	Flat Terrain	The ground is assumed perfectly flat and horizontal. The 2D grid model ignores elevation changes. Stability is evaluated purely based on load distribution, not terrain slope.
A3	Rigid Body	The robot body is modelled as a rigid structure — no joint compliance or body flex. The four foot positions are fixed relative to the body centre at all times during stance.
A4	Quasi-Static Motion	The robot moves slowly enough that dynamic effects (inertia, acceleration) are negligible. Stability is evaluated at each static stance position, not during the swing phase.
A5	Payload Fixed to Body	The payload is rigidly attached to the robot body at a known position. Its mass and position do not change during navigation. The payload CoM is assumed known prior to planning.
A6	4-Connected Movement	The robot moves in four cardinal directions only (up, down, left, right). Diagonal movement is not considered. Each grid cell is exactly 1.0 m in physical size.

These assumptions define the scope of this study. Real-world extensions — dynamic obstacles, sloped terrain, joint compliance, diagonal movement — are left as future work and will be noted in the paper's Conclusion section.

9. Observations & Notes

The hardest part was getting the CoM equation right in words: The code computes it correctly, but writing it for a paper audience required being explicit about the approximation

— we approximate the body CoM as the geometric centroid of the foot positions, not a full inertial model. This is a deliberate simplification that should be stated clearly so reviewers understand the scope.

Assumption A4 (quasi-static) is the most important one: Almost everything in the stability analysis assumes the robot is standing still when we evaluate S(n) and M(n). If the robot is mid-stride, the support polygon changes completely. Noting this clearly prevents reviewers from asking obvious questions about dynamic stability.

The notation table will save a lot of time in the Results section: Every symbol is defined here once. When writing results like "Avg S(n) = 0.22", the reader can immediately look up what S(n) means without hunting through the text.